RAG

(Retrieval-Augmented Generation)

Complete Developer Study Notes

Concepts • Practical Examples • Projects • Interview Preparation

Prepared by: Rahul Yadav

20 Pages

Rahul Yadav

1. Introduction to RAG

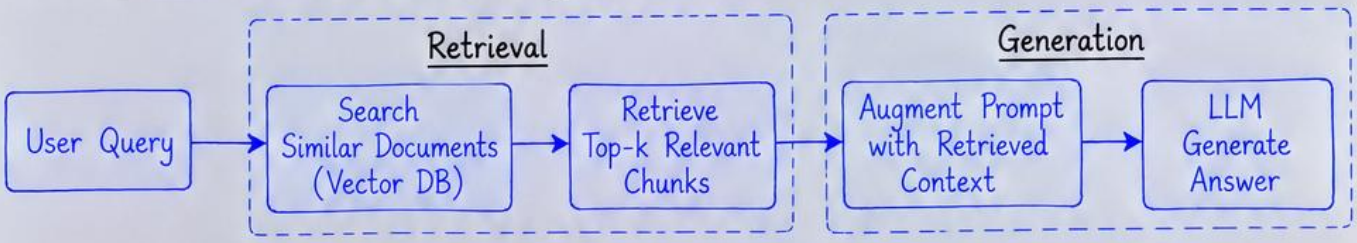
1.1 What is RAG ?

- RAG (Retrieval-Augmented Generation) is a technique that combines information retrieval with large language models (LLMs) to generate more accurate, relevant and up-to-date responses.
- Instead of relying only on the model's internal knowledge, RAG first retrieves relevant information from external data sources (like documents, databases, websites) and then uses this information to generate an answer.

1.2 Why RAG is Important ?

- LLMs have a fixed knowledge cutoff and may not know the latest information.
- They can sometimes generate incorrect or hallucinated answers.
- RAG allows models to use external, real-world data to provide more accurate, reliable and domain-specific responses.
- Useful for enterprise applications, document Q&A, research, customer support, and knowledge management.

1.3 How RAG Works ? (High Level)



1.4 Key Components of RAG

Component	Description
Data Source	Documents, PDFs, websites, databases, APIs, etc.
Document Processing	Load, clean, split and create chunks from documents.
Embeddings	Convert text chunks into vector representations.
Vector Database	Store and search embeddings (FAISS, Qdrant, Pinecone, etc.)
Retriever	Find top-k similar chunks for a user query.
Prompt Augmentation	Combine query + retrieved context into a prompt.
LLM	Generate final answer using augmented prompt.

1.5 Real-world Use Cases

- Chat with your documents (PDF, Docs, internal knowledge base)
- Customer support chatbot using company data
- Research assistant for academic papers
- Enterprise knowledge search (policies, manuals, documentation)
- Domain-specific Q&A (finance, healthcare, legal, etc.)
- Personalized search and recommendations

Rahul Yadav

2. Limitations of LLMs and Need for RAG

2.1 Limitations of LLMs

- **Knowledge Cutoff**: LLMs are trained on data up to a certain date and do not know the latest information.
- **Hallucinations**: Sometimes LLMs generate incorrect or made-up information that sounds reasonable but is not true.
- **No Access to Private/Company Data**: LLMs cannot access your internal documents, databases or real-time data.
- **Limited Source Traceability**: It is difficult to know where the information came from.
- **Context Window Limitations**: LLMs have a limited context length, so they cannot process very large documents directly.
- **Domain-specific Accuracy**: Performance may be poor for niche or domain-specific topics without additional context.

2.2 LLM Alone vs RAG (High Level Comparison)

LLM Alone



- Uses only internal knowledge
- May return outdated or incorrect information
- Can hallucinate
- No source reference

RAG (Retrieval-Augmented Generation)

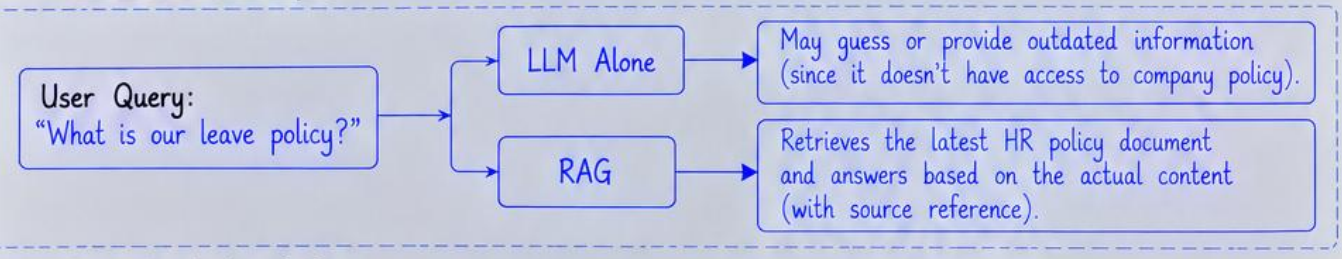


- Retrieves relevant information from external sources
- Combines retrieved context with user query
- Generates a more accurate and up-to-date answer
- Can provide source references

2.3 How RAG Helps to Overcome Limitations

Challenge	How RAG helps
Outdated knowledge	Retrieves the latest information from external documents, databases or websites.
Private or internal data	Can use company documents, policies, knowledge base, etc.
Hallucinations and incorrect answers	Provides relevant context, reducing the chance of hallucinations.
Source verification	Can return the source documents used for the answer.
Domain-specific questions	Retrieves domain-specific documents to give accurate and relevant responses.

2.4 Practical Example



2.5 Important Limitation

- RAG can still fail if the retrieved documents are stale or incorrect.
- If retrieval does not find the right information, the answer may still be wrong.
- The model may sometimes ignore the provided context.
- RAG reduces hallucinations but does not eliminate them completely.

2.6 Key Takeaways

- LLMs are powerful but have limitations.
- RAG helps by retrieving relevant, real-world context at query time.
- Use high-quality and up-to-date documents for better results.

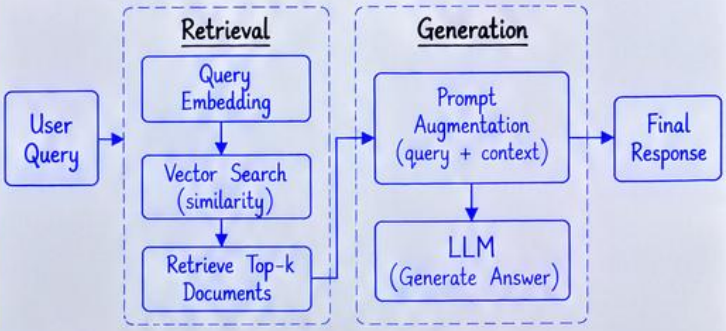
Rahul Yadav

3. RAG Architecture and Workflow

3.1 Overview

- RAG combines information retrieval with generative models to provide more accurate and up-to-date answers.
- It retrieves relevant information from external data sources and uses that context to generate a response.
- This architecture helps reduce hallucinations and allows LLMs to use private or domain-specific data.
- RAG works in a modular pipeline with multiple components.

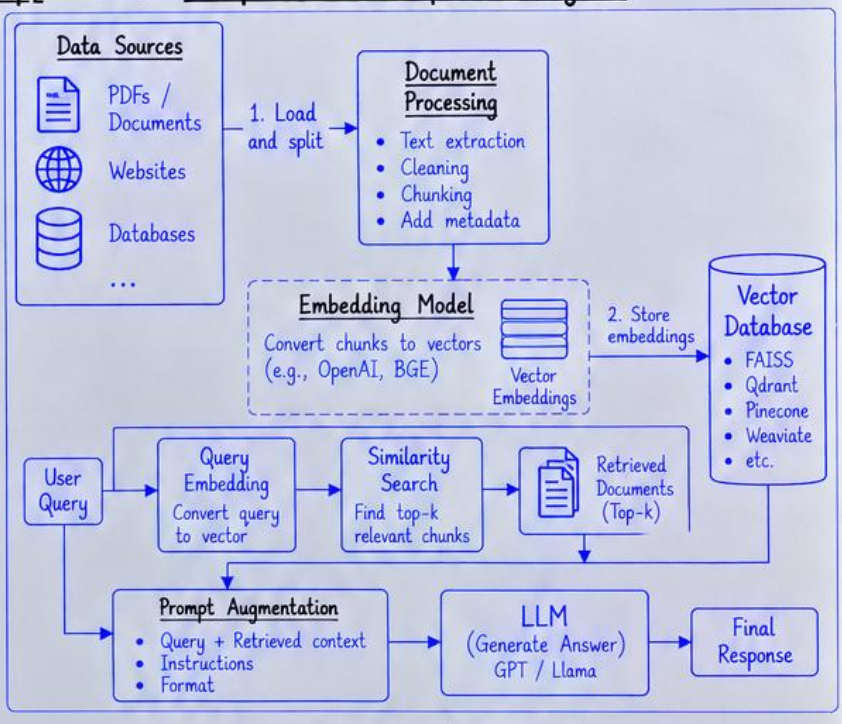
3.2 High Level Architecture



3.3 Detailed Workflow (Step by Step)

1. **User Query**
   User asks a question in natural language.
2. **Query Embedding**
   Convert the query into a vector using an embedding model (e.g., OpenAI, BGE).
3. **Document Retrieval**
   Search similar documents from a vector database using the query embedding.
4. **Context Preparation**
   Select top-k relevant chunks and format them as context for the LLM.
5. **Prompt Augmentation**
   Combine the user query with retrieved context using a prompt template.
6. **Answer Generation**
   LLM generates a final answer based on the provided context.

3.4 Complete RAG Pipeline Diagram



3.5 Prompt Template Example

```
You are a helpful assistant. Use the following context to
answer the user's question. If the answer is not in the
context, say you don't know.
Context:
{retrieved_documents}
-----------------------------------
Question: {user_query}
Answer:
```

3.6 Example Flow (HR Policy)



3.7 Key Components Summary

Component	Description
Data Loader	Load data from PDF, websites, databases, APIs
Text Splitter	Split documents into smaller chunks
Embedding Model	Convert text to vector embeddings
Vector Database	Store and search embeddings efficiently
Retriever	Find top-k relevant chunks for a query
Prompt Augmentation	Combine query + context into a prompt
LLM	Generate final answer using the augmented prompt

3.8 Benefits of RAG Architecture

- Uses real and up-to-date information
- Reduces hallucinations
- Can work with private and domain-specific data
- Provides source references
- Improves accuracy and trustworthiness
- Scalable and modular architecture

3.9 Common Use Cases

- Company knowledge base (HR, policies, documentation)
- Customer support chatbot
- Research and academic Q&A
- Enterprise search
- Legal, healthcare, finance domain applications

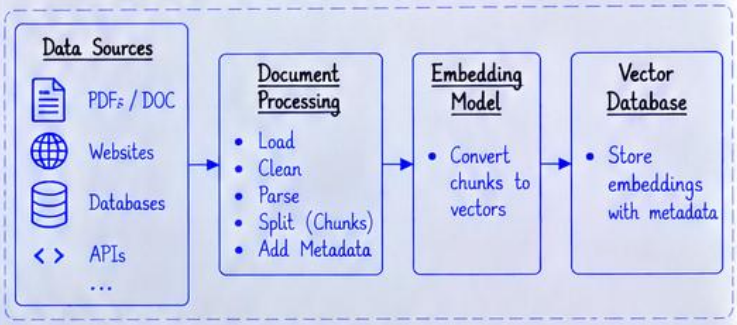
Rahul Yadav

4. Document Ingestion and Processing

4.1 Overview

- Document ingestion is the process of loading data from different sources (PDF, DOC, websites, databases, APIs, etc.) into the RAG pipeline.
- The documents are then processed, cleaned, split into smaller chunks and converted into vector embeddings for retrieval.
- Good document processing is very important because the quality of retrieved results depends on how well we ingest and prepare the documents.

4.2 Ingestion Pipeline (High Level)



4.3 Supported Document Sources

Source Type	Examples
Documents	PDF, DOCX, TXT, PPTX, XLSX
Webpages	Web URLs, Blogs, Documentation
Databases	MySQL, PostgreSQL, MongoDB
APIs	REST APIs, Internal APIs
Cloud Storage	S3, Google Drive, SharePoint
Others	Emails, Notion, Confluence, etc.

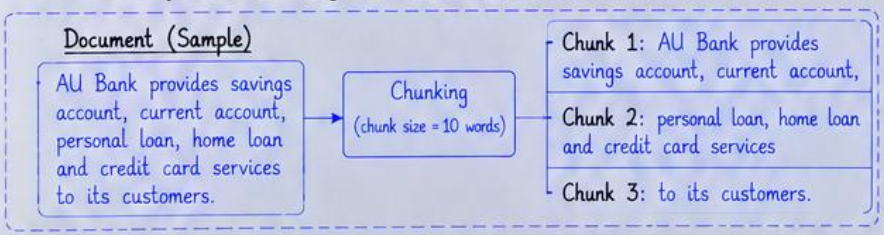
4.4 Document Processing Steps

1. **Load Documents**
   Read the file from different sources using appropriate loaders.
2. **Text Extraction**
   Extract text from the document (handle different formats).
3. **Cleaning**
   Remove unwanted content (headers, footers, extra spaces, special characters, etc.).
4. **Chunking**
   Split the text into smaller chunks using fixed size or semantic chunking.
5. **Add Metadata**
   Add useful metadata (source, page number, chunk id, etc.).
6. **Generate Embeddings**
   Convert each chunk into a vector using an embedding model.
7. **Store in Vector DB**
   Store the embeddings along with metadata for similarity search.

4.5 Chunking Strategies

Strategy	Description
Fixed Size Chunking	Split text into fixed number of tokens (e.g., 500 tokens).
Recursive Chunking	Split based on document structure (headings, paragraphs).
Semantic Chunking	Split based on meaning using embeddings.
Sliding Window	Create overlapping chunks to maintain context.

4.6 Example: Chunking a Document



4.7 Adding Metadata

```
{
    "source": "au_bank_policy.pdf",
    "page_number": 5,
    "chunk_id": "chunk_1",
    "title": "Account Types",
    "department": "Banking",
    "file_type": "pdf"
}
```

4.8 Code Example (LangChain - Document Loading and Splitting)

```
from langchain.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter

# Load document
loader = PyPDFLoader("au_bank_policy.pdf")
documents = loader.load()

# Split into chunks
splitter = RecursiveCharacterTextSplitter(
    chunk_size = 500,
    chunk_overlap = 50
)
chunks = splitter.split_documents(documents)
```

4.9 Best Practices

- Use appropriate loader for each file type.
- Clean and preprocess the text.
- Choose the right chunk size (300-1000 tokens).
- Add meaningful metadata.
- Use overlapping chunks.
- Handle tables and structured data properly.
- Test retrieval quality with different chunking strategies.

4.10 Common Challenges

- Handling complex document formats (tables, images).
- Removing irrelevant or noisy content.
- Choosing the right chunk size.
- Maintaining context across chunks.
- Processing large documents efficiently.

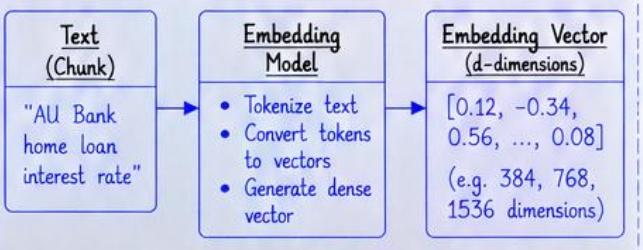
Rahul Yadav

5. Text Embeddings and Vector Representations

5.1 What are Text Embeddings?

- Text embeddings are numerical vector representations of text that capture semantic meaning.
- Similar texts have similar vectors and different texts have different vectors.
- They help in finding relevant documents using similarity search.
- Embeddings convert human language into a format that machines can understand and compute.

5.2 How Text is Converted to Embeddings?



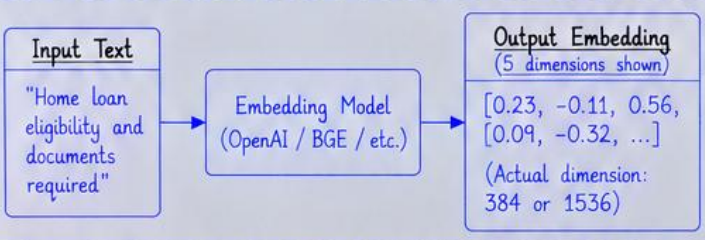
5.3 Popular Embedding Models

Model	Dimensions	Highlights / Use Cases
OpenAI (text-embedding-3-small)	1536	High quality, general purpose
OpenAI (text-embedding-3-large)	3072	Better performance, semantic search
BGE (BAAI/bge-small-en)	384	Good for RAG, open source
BGE (bge-base-en)	768	Better accuracy
Sentence Transformers (all-MiniLM-L6-v2)	384	Fast, lightweight, popular
Instructor Embeddings	768	Good for domain-specific tasks
Cohere Embeddings	1024	Multilingual support

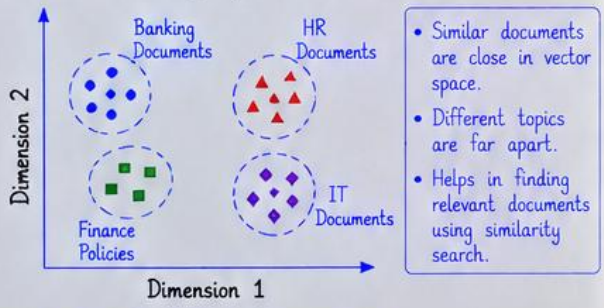
5.4 Embedding Process (Detailed)

1. **Text Preprocessing**
   - Clean the text (remove extra spaces, special chars)
   - Split into chunks if required
2. **Tokenization**
   - Break text into tokens (words / subwords)
3. **Convert to Vectors**
   Each token is converted into a numerical vector using a neural model.
4. **Pooling**
   Combine token vectors (mean pooling or special token) to get a single vector for the text.
5. **Normalization (Optional)**
   Normalize the vector to unit length for better similarity search.

5.5 Example: Text to Embedding



5.6 Embedding Space Visualization



5.7 Similarity Search

- Once text is converted into embeddings, we can find similar documents using distance metrics.
- Common similarity metrics:

Metric	Formula / Description	Use Case
Cosine Similarity	Measures angle between vectors	Most common for text embeddings
Euclidean Distance	Straight line distance	Sometimes used
Dot Product	Product of vectors	Used in some models

Cosine Similarity Formula:

$$\text{sim}(A, B) = \frac{(A \cdot B)}{||A||\ ||B||}$$

- Value range: -1 to 1
- 1 = very similar
- 0 = no similarity
- -1 = completely different

5.8 Code Example (OpenAI Embeddings)

```
from openai import OpenAI
client = OpenAI(api_key="your-api-key")

text = "Home loan eligibility and documents required"
response = client.embeddings.create(
    model="text-embedding-3-small",
    input=[text]
)
embedding = response.data[0].embedding
print(len(embedding))   # 1536
print(embedding[:5])    # first 5 values
```

5.9 Key Takeaways

- Text embeddings capture the semantic meaning of text.
- Similar texts have similar vectors.
- Embeddings are essential for efficient document retrieval in RAG.
- Choose the right embedding model based on your use case.
- Use cosine similarity for better search results.
- Higher quality embeddings improve RAG accuracy.

5.10 Common Challenges

- Choosing the right embedding model.
- Handling domain-specific or technical content.
- Long text embeddings (use chunking).
- Multilingual embeddings for different languages.
- Keeping embeddings up-to-date when documents change.

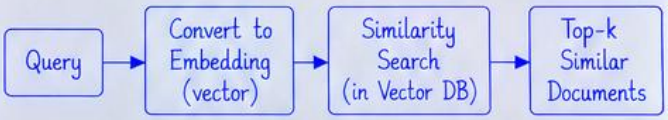
Rahul Yadav

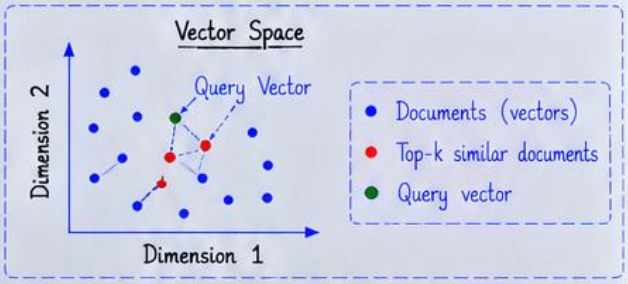
6. Vector Databases for RAG

6.1 What is a Vector Database ?

- A vector database is a specialized database designed to store, index and search high-dimensional vector embeddings.
- It enables fast similarity search to find relevant documents based on semantic meaning.
- Instead of exact keyword matching, it finds similar vectors using distance metrics.
- Vector databases are a key component in RAG for efficient retrieval of relevant context.

6.2 How Vector Search Works ?





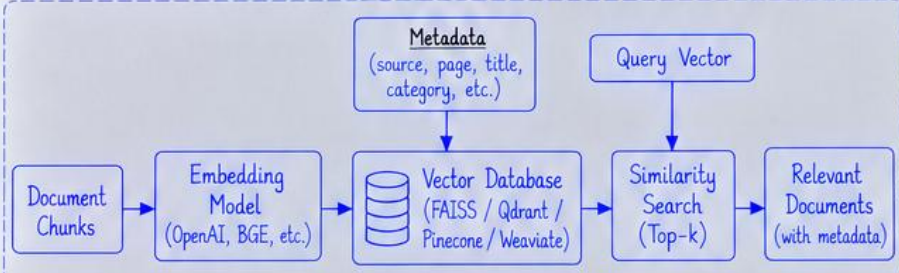
6.3 Popular Vector Databases

Database	Type	Key Features	Use Cases
FAISS	In-memory (Library)	Fast, efficient similarity search	Research, small/medium data
Qdrant	Open-source	Filtering, payload, scalable, easy to use	Production RAG applications
Pinecone	Managed (Cloud)	Fully managed, high scalability	Enterprise applications
Weaviate	Open-source	Graph + vector, hybrid search	Semantic search, knowledge graphs
Chroma	Open-source	Simple, lightweight	Prototyping, small projects
MilVus	Open-source	Highly scalable, distributed	Large scale applications

6.4 Similarity Search Algorithms

Algorithm	Description	Pros	Cons
Cosine Similarity	Measures angle between vectors	Most common, works well for text	Not good for magnitude
Euclidean Distance	Straight line distance	Simple and intuitive	Not ideal for high dimensions
Dot Product	Product of vectors	Fast, good for normalized vectors	Sensitive to vector magnitude
Approximate Nearest Neighbor (ANN)	Finds similar vectors efficiently (FAISS, HNSW)	Very fast for large datasets	May not always return exact nearest neighbor

6.5 Vector Database Architecture



6.6 Storing Vectors with Metadata

```
{
    "id": "doc_1",
    "vector": [0.12, -0.34, 0.56, ...],
    "metadata": {
        "source": "au_bank_policy.pdf",
        "page": 5,
        "title": "Leave Policy",
        "department": "HR",
        "chunk_id": "chunk_1",
        "file_type": "pdf"
    }
}
```

6.7 Code Example (Using Qdrant)

```
from qdrant_client import QdrantClient
from qdrant_client.models import Distance, VectorParams, PointStruct
# 1. Create client
client = QdrantClient(":memory:")
# 2. Create collection
client.recreate_collection(
    collection_name="rag_docs",
    vectors_config=VectorParams(size=1536, distance=Distance.COSINE)
)
# 3. Insert document vector with metadata
client.upsert(
    collection_name="rag_docs",
    points=[
        PointStruct(
            id="doc_1",
            vector= [0.12, -0.34, 0.56, ...],
            payload={"source": "au_bank_policy.pdf", "page": 5, "title": "Leave Policy"}
        )
    ]
)
```

6.8 Querying the Vector Database

```
# 1. Generate query embedding using OpenAI
query_vector = embedding_model.embed_query("What is our leave policy?")
# 2. Search similar documents
search_result = client.search(
    collection_name="rag_docs",
    query_vector=query_vector,
    limit=5
)
# 3. Print results
for res in search_result:
    print(res.payload["source"], res.payload["page"], res.score)
```

6.9 Best Practices

- Choose the right vector database based on your scale and needs.
- Store useful metadata for better filtering.
- Use appropriate embedding model and vector dimension.
- Use hybrid search (vector + keyword) for better accuracy.
- Regularly update and maintain the vector index.

Rahul Yadav

7. Retrieval Techniques for RAG

7.1 Overview

- Retrieval is the process of finding the most relevant documents or document chunks from a vector database based on the user query.
- Good retrieval techniques improve the quality, relevance and accuracy of the final answer in RAG.
- Different retrieval strategies are used depending on the use case, data type and requirements.

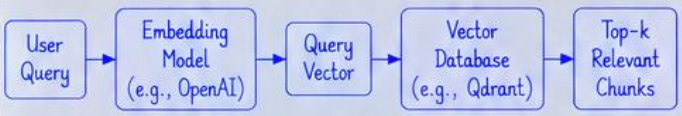
7.2 Types of Retrieval Techniques

Similarity Search	• Finds documents similar to the query using vector embeddings. • Most common technique in RAG.
Keyword Search (Lexical Search)	• Uses exact keyword matching (BM25, etc.). • Good for precise keyword queries.
Hybrid Search	• Combines semantic (vector) and lexical (keyword) search. • Provides better and more accurate results.

7.3 Similarity Search (Vector Search)

- Converts the user query into an embedding vector.
- Calculates similarity between the query vector and document vectors.
- Returns top-k most similar documents.
- Uses similarity metrics like cosine similarity, euclidean distance or dot product.

Similarity Search Flow



7.4 Keyword Search (BM25)

- BM25 is a popular lexical search algorithm.
- It ranks documents based on keyword relevance using term frequency and inverse document frequency.
- Useful when exact keywords or domain-specific terms are important.

BM25 Scoring Formula

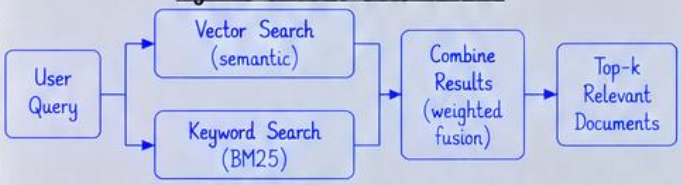
$$\text{score}(D, Q) = \sum_{i=1}^{n} \text{IDF}(q_i) \cdot \frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{\text{avgDL}}\right)}$$

- q_i : query term
- $f(q_i, D)$: term frequency in document D
- $\text{IDF}(q_i)$: inverse document frequency
- k_1, b : parameters (usually $k_1 = 1.5$, $b = 0.75$)

7.5 Hybrid Search

- Combines vector search (semantic) and keyword search (lexical).
- Helps to get more relevant and comprehensive results.
- Useful when documents contain both technical terms and natural language.

Hybrid Search Architecture



7.6 Comparison of Retrieval Techniques

Technique	Method	Pros	Cons	Best Use Case
Similarity Search	Vector Embeddings	Good semantic understanding	May miss exact keywords	General RAG applications
Keyword Search (BM25)	Lexical Matching	Good for exact keywords	No semantic understanding	Domain-specific keywords
Hybrid Search	Vector + Keyword	Better accuracy and recall	Slightly more complex	Most real-world use cases

7.7 Code Example (Vector Search using Qdrant)

```python
from qdrant_client import QdrantClient
from qdrant_client.models import VectorParams, Distance
from openai import OpenAI

# 1. Create client
client = QdrantClient("localhost", port=6333)

# 2. Create collection (if not exists)
client.recreate_collection(
    collection_name="rag_docs",
    vectors_config=VectorParams(size=1536, distance=Distance.COSINE)
)

# 3. Search similar documents
query = "What is our leave policy?"
query_vector = OpenAI(api_key="your-api-key").embeddings.create(
    model="text-embedding-3-small",
    input=[query]
).data[0].embedding

search_result = client.search(
    collection_name="rag_docs",
    query_vector=query_vector,
    limit=5
)
for res in search_result:
    print(res.payload["content"], res.score)
```

7.8 Advanced Retrieval Techniques

- **Re-ranking**: Re-rank the retrieved results using a cross-encoder model for better relevance.
- **Query Expansion**: Expand the user query with related terms or synonyms.
- **Metadata Filtering**: Use metadata (e.g., department, document type) to filter results.
- **MMR (Maximal Marginal Relevance)**: Improve diversity in results by reducing redundant documents.
- **Multi-query Retrieval**: Generate multiple versions of the query using an LLM and retrieve results for each.

7.9 Best Practices

- Choose the right retrieval technique based on your data and use case.
- Use hybrid search for better accuracy.
- Apply metadata filtering to narrow down results.
- Use re-ranking for high-quality answers.
- Tune the value of top-k (e.g., 3–10).
- Evaluate retrieval quality using metrics like Recall@K, MRR, etc.

7.10 Key Takeaways

- Retrieval techniques are crucial for the success of RAG.
- Vector search uses semantic similarity, keyword search uses lexical matching, and hybrid search combines both.
- Advanced techniques like re-ranking and query expansion further improve results.
- Always evaluate and tune your retrieval strategy for better accuracy.

8. Prompt Engineering for RAG

8.1 What is Prompt Engineering for RAG?

- Prompt engineering is the process of designing effective prompts to guide the LLM to use retrieved context and generate accurate answers.
- In RAG, we provide the LLM with retrieved documents (context) along with a well-structured prompt.
- Good prompts improve relevance, reduce hallucinations and ensure grounded responses.

8.2 Importance of Prompt Engineering

- Helps the LLM understand how to use the context.
- Reduces irrelevant or incorrect answers.
- Controls the response style, length and format.
- Can include instructions, examples and constraints.
- Very important for domain-specific applications (e.g., HR policy, finance, legal).

8.3 Basic RAG Prompt Template

```
You are a helpful assistant. Use the following context
to answer the question. If the answer is not in the context,
say you don't know.
Context:
{context}
Question: {user_question}
Answer:
```

8.4 Prompt Template Variations

Type	Template Focus	When to Use
Zero-shot	Simple instruction with context	General use cases
Few-shot	With examples	Complex questions, better accuracy
Chain-of-Thought	Step-by-step reasoning	Analytical or multi-step questions
Structured Output	Ask for specific format (JSON, bullet points)	When you need defined output format
Role-based Prompt	Define assistant role (e.g., HR Expert)	Domain-specific applications

8.5 Example Prompts

1. Simple Prompt

```
Context:
{context}

Question: {question}

Answer:
```

2. Prompt with Instructions

```
You are an HR policy expert.
Use the context to answer the
question. Provide a clear and
concise answer in simple language.
If the answer is not in the context,
say "I don't know".
Context: {context}
Question: {question}
Answer:
```

3. Prompt with Format (Bullet Points)

```
You are a helpful assistant. Answer the question
using the given context. Provide the answer in
bullet points.
Context: {context}
Question: {question}
Answer:
```

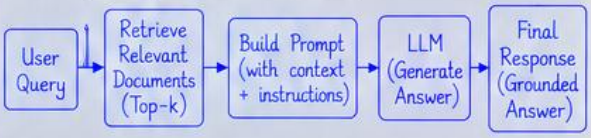
4. Prompt with Example (Few-shot)

```
Example:
Q: What is the leave policy?
A: Employees get 20 days of paid leave per year.
Now answer the following question using the context.
Context: {context}
Question: {question}
Answer:
```

8.6 Prompt Engineering Techniques

Technique	Description	Example / Use Case
Clear Instructions	Tell the model what to do	Answer using context only
Provide Context	Include relevant chunks	Top-k retrieved documents
Use Examples	Give input-output examples	Few-shot prompting
Specify Output Format	Define format (JSON, bullets)	Structured responses
Role Prompting	Assign a role to the LLM	HR expert, legal assistant
Control Length	Ask for concise answers	Max 100 words
Handle Unknown	Instruct to say "I don't know"	Avoid hallucinations
Use Chain-of-Thought	Encourage step-by-step reasoning	Complex analytical queries

8.7 Prompt Flow in RAG



8.9 Example: Using Prompt with LLM (LangChain)

```python
from langchain.chains import RetrievalQA
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model="gpt-4o", temperature=0)
qa_chain = RetrievalQA.from_chain_type(
    llm=llm,
    retriever=vector_store.as_retriever(search_kwargs={"k": 5}),
    chain_type="stuff",
    chain_type_kwargs={"prompt": prompt_template}
)

response = qa_chain.invoke({"query": "What is the leave policy?"})
print(response["result"])
```

8.8 Code Example (LangChain PromptTemplate)

```python
from langchain.prompts import PromptTemplate
prompt_template = PromptTemplate(
    input_variables=["context", "question"],
    template="""
    You are a helpful assistant. Use the following context
    to answer the question. If the answer is not in the context,
    say you don't know.
    {context: {context}
    Question: {question}
    Answer: """)
```

8.10 Best Practices

- Use clear and specific instructions.
- Include only relevant context (avoid too large context).
- Experiment with different templates for better results.
- Use few-shot examples for complex queries.
- Control the response format and length.
- Always handle cases when the answer is not in the context.
- Iterate and evaluate prompts for your use case.

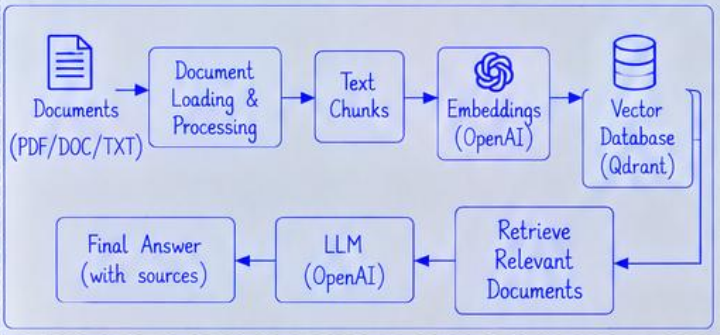
Rahul Yadav

9. RAG Implementation (Hands-on)

9.1 Overview

- In this section, we will build a simple end-to-end RAG application using real documents.
- We will use Python, LangChain, OpenAI, and Qdrant to implement the complete pipeline.
- The application will load documents, create embeddings, store them in a vector database and retrieve relevant documents to generate accurate answers.

9.2 Project Architecture



9.3 Tech Stack Used

Component	Tool / Library	Purpose
Language	Python	Implementation
Framework	LangChain	RAG pipeline and orchestration
Embedding Model	OpenAI (text-embedding-3-small)	Convert text to vectors
Vector Database	Qdrant	Store and search embeddings
LLM	OpenAI (gpt-4o / gpt-3.5)	Generate final answer
Document Loader	LangChain loaders	Load PDF, DOCX, TXT, etc.
Text Splitter	RecursiveCharacterTextSplitter	Split documents into chunks

9.4 Step-by-Step Implementation

1. Load and process documents (PDF/ TXT / DOCX).
2. Split documents into smaller chunks.
3. Generate embeddings using OpenAI.
4. Store embeddings in Qdrant vector database.
5. Retrieve relevant documents for a user query.
6. Generate final answer using LLM with retrieved context.

9.5 Code Example - Complete RAG Pipeline

```
# 1. Install required libraries
# pip install langchain langchain-openai langchain-qdrant qdrant-client

from langchain.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings, ChatOpenAI
from langchain_qdrant import QdrantVectorStore
from qdrant_client import QdrantClient
from langchain.chains import RetrievalQA

# 2. Load document
loader = PyPDFLoader("au_bank_policy.pdf")
documents = loader.load()

# 3. Split into chunks
splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=50
)
chunks = splitter.split_documents(documents)

# 4. Generate embeddings
embeddings = OpenAIEmbeddings(model="text-embedding-3-small")

# 5. Store in Qdrant
client = QdrantClient("localhost", port=6333)
vector_store = QdrantVectorStore(
    client=client,
    collection_name="rag_docs",
    embedding=embeddings
)
vector_store.add_documents(chunks)

# 6. Create Retrieval QA chain
retriever = vector_store.as_retriever(search_kwargs={"k": 5})
llm = ChatOpenAI(model="gpt-4o", temperature=0)
qa_chain = RetrievalQA.from_chain_type(
    llm=llm,
    retriever=retriever,
    chain_type="stuff"
)

# 7. Query
query = "What is the leave policy at AU Bank?"
response = qa_chain.invoke({"query": query})
print(response["result"])
```

9.6 Sample Output

```
Query: What is the leave policy at AU Bank?
Answer:
Employees are entitled to 20 days of paid leave per year
as per the AU Bank HR policy. This includes casual leave,
sick leave and optional leave.

Sources:
1. au_bank_policy.pdf (Page 5)
2. leave_policy_2024.pdf (Page 2)
```

9.7 Adding Metadata

```
{
    "source": "au_bank_policy.pdf",
    "page": 5,
    "title": "Leave Policy",
    "department": "HR",
    "chunk_id": "chunk_1",
    "file_type": "pdf"
}
```

9.8 Testing with Different Queries

Query	Expected Output
What is the leave policy?	Leave rules with source documents
How many casual leaves are allowed?	Exact number from policy document
What is the reimbursement process?	Steps from relevant document
Who can apply for home loan in AU Bank?	Answer from HR or finance document

9.9 Best Practices

- Use clean and well-structured documents.
- Choose appropriate chunk size (300-1000 tokens).
- Add useful metadata (source, page, title, etc.).
- Use hybrid retrieval for better results.
- Evaluate responses and handle cases when answer is not found.
- Use caching to reduce cost and improve performance.

Rahul Yadav

10. Advanced Retrieval Techniques

10.1 Overview

- Basic retrieval (similarity search) works well for simple queries, but advanced techniques are needed for complex, multi-hop and domain-specific queries.
- Advanced retrieval techniques improve the relevance, diversity and accuracy of retrieved documents.
- These techniques are often combined in real-world RAG applications to get better results.

10.2 Query Expansion

- Query expansion rewrites the user query by adding related terms, synonyms or context to improve retrieval.
- It helps to find more relevant documents even if the exact keywords are not present in the documents.
- We can use LLM or predefined synonym lists for query expansion.

Example

Original Query: What is leave policy?
Expanded Query: What is leave policy, paid leave, vacation, casual leave, sick leave, holiday policy

10.3 Query Expansion using LLM (Code)

```
from langchain_openai import ChatOpenAI
from langchain.prompts import PromptTemplate

llm = ChatOpenAI(model="gpt-4o", temperature=0)

prompt = PromptTemplate(
    input_variables=["query"],
    template="""
    Generate 3-5 related search queries or synonyms for the following query. Return only the queries in a list.
    Query: {query}
    """
)

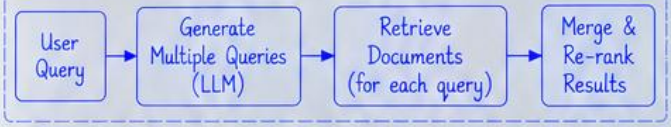
chain = prompt | llm

result = chain.invoke({"query": "What is leave policy?"})
print(result.content)
```

10.4 Multi-Query Retrieval

- Generate multiple related queries using LLM.
- Retrieve documents for each query.
- Combine and deduplicate the results.
- Improves recall by covering different aspects of the question.

Multi-Query Retrieval Flow



10.5 Step-Back Retrieval

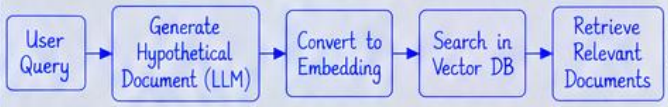
- It generates a more general version of the query (step-back query) to retrieve broader context, and then uses the original query for final answer.
- Helpful for complex or specific questions.

Original Query: How many days of paid leave are allowed?
Step-back Query: What is the leave policy in the company?

10.6 Hypothetical Document Embeddings (HyDE)

- Generates a hypothetical document using the query.
- Embeds this document and uses it to search similar documents.
- Useful when the query is too short or ambiguous.

HyDE Process



10.7 Metadata Filtering

- Use metadata (e.g., department, date, document type) to filter documents.
- Helps to restrict search to relevant data.
- Useful in enterprise applications (e.g., HR, Finance).

Example: Filter by department

```
search_result = vector_store.similarity_search(
    query="What is the leave policy?",
    filter={"department": "HR", "doc_type": "policy"},
    k=5
)
```

10.8 Re-ranking Techniques

- Re-ranking improves the order of retrieved results using a cross-encoder or LLM.
- It considers the query and document together to calculate relevance score.
- Provides more accurate and contextually relevant results.

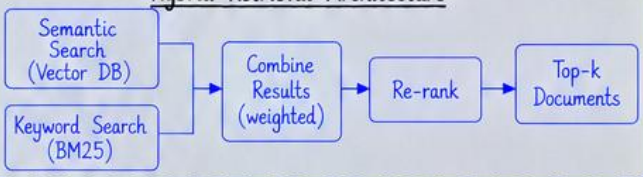
Popular Re-ranking Models

- Cross-Encoder models (e.g., bge-reranker, Cohere Rerank)
- LLM-based re-ranking
- Sentence Transformers

10.9 Hybrid Retrieval (Advanced)

- Combines multiple retrieval techniques (e.g., semantic search + BM25 keyword search + metadata filtering + re-ranking).
- Gives better recall and precision.
- Common in production RAG systems.

Hybrid Retrieval Architecture



10.10 Best Practices

- Use query expansion for short or ambiguous queries.
- Combine multiple retrieval techniques (hybrid search).
- Apply metadata filtering to narrow down results.
- Use re-ranking for better accuracy.
- Experiment and evaluate using real user queries.
- Monitor retrieval quality and continuously improve.

10.11 Key Takeaways

- Advanced retrieval techniques significantly improve RAG performance.
- Combine techniques based on your use case and data.
- Hybrid search + re-ranking is widely used in real-world applications.
- Proper evaluation is important to measure retrieval quality.

Rahul Yadav

11. RAG Optimization and Evaluation

11.1 Overview

- Optimizing RAG is important to get accurate, relevant and grounded answers.
- Evaluation helps to measure the quality of retrieval and generation.
- In this section, we will learn different techniques to optimize RAG and how to evaluate its performance using various metrics.

11.2 Why Optimization is Needed ?

- Improve retrieval accuracy and relevance.
- Reduce irrelevant or noisy results.
- Handle complex and domain-specific queries.
- Improve response quality and reduce hallucinations.
- Make RAG faster and more efficient for production use cases.

11.3 Areas of Optimization

- Document ingestion and preprocessing.
- Chunking strategy and metadata design.
- Embedding model and vector database settings.
- Retrieval techniques (hybrid search, re-ranking).
- Prompt engineering and response generation.
- System performance (latency, scalability, cost).

11.4 Chunking Optimization

- Choose the right chunk size (e.g., 200-500 tokens).
- Use overlapping chunks (e.g., 50-100 tokens).
- Keep semantic meaning in each chunk.
- Add useful metadata (document type, section, etc.).

11.5 Example: Optimized Text Splitter (LangChain)

```
from langchain.text_splitter import RecursiveCharacterTextSplitter
text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=100,
    separators=["\n\n", "\n", ".", ",", " "]
)
chunks = text_splitter.split_text(document)
```

11.6 Retrieval Optimization Techniques

- Use hybrid search (semantic + keyword).
- Apply metadata filtering.
- Use re-ranking models to improve relevance.
- Adjust top-k value based on use case.
- Use query expansion for better recall.
- Tune embedding model based on domain (OpenAI, BGE, etc.).

11.7 Example: Hybrid Retrieval (LangChain)

```
from langchain.retrievers import EnsembleRetriever
retriever = EnsembleRetriever(
    retrievers=[vector_retriever, bm25_retriever],
    weights=[0.7, 0.3]
)
```

11.8 Generation Optimization

- Use clear and well-structured prompts.
- Provide context in a readable format.
- Limit the number of retrieved chunks (e.g., top 3-5).
- Use chain-of-thought for complex questions.
- Control response length and format (bullets, JSON, etc.).
- Include "I don't know" instruction to avoid hallucinations.

11.9 Evaluation Metrics

- We evaluate both retrieval quality and generation quality.
- Common metrics for RAG evaluation:
  - Precision@K – How many retrieved docs are relevant
  - Recall@K – How many relevant docs are retrieved
  - MRR (Mean Reciprocal Rank) – Rank of first relevant doc
  - BLEU / ROUGE – Text similarity (for generation)
  - RAGAS – RAG-specific evaluation framework
  - Faithfulness – Whether the answer is grounded in context
  - Relevance – How relevant the answer is to the question
  - Context Recall – Whether all relevant information is used

11.10 Precision@K and Recall@K

- Precision@K = Relevant documents in top K / K
- Recall@K = Relevant documents in top K / Total relevant documents

Example:

K = 5
Relevant documents in top 5 = 4
Total relevant documents = 10
Precision@5 = 4 / 5 = 0.8
Recall@5 = 4 / 10 = 0.4

11.11 Mean Reciprocal Rank (MRR)

- MRR measures the rank of the first relevant document.
- Formula: MRR = 1 / Rank of first relevant document
- If the first relevant document is at rank 2, MRR = 1/2 = 0.5

11.12 RAGAS Evaluation

- RAGAS is a popular framework to evaluate RAG systems.
- It provides metrics like faithfulness, answer relevance, context recall and context precision.

```
from ragas import evaluate
from ragas.metrics import faithfulness, answer_relevancy,
      context_recall, context_precision
result = evaluate(
     dataset,
     metrics=[faithfulness, answer_relevancy,
              context_recall, context_precision]
)
print(result)
```

11.13 Best Practices

- Use high-quality and clean documents.
- Experiment with different chunk sizes and top-k values.
- Use hybrid retrieval with re-ranking.
- Evaluate using multiple metrics (retrieval + generation).
- Monitor real user queries and feedback.
- Continuously tune and improve the pipeline.

11.14 Key Takeaways

- Optimization improves the accuracy, relevance and efficiency of RAG.
- Evaluation helps measure the performance using standard metrics.
- Use a combination of retrieval, generation and system-level optimization to get the best results.

Rahul Yadav

12. RAG with Different Data Sources

12.1 Overview

- RAG can work with multiple data sources like PDF files, text files, websites, databases, APIs, and more.
- Each data source needs a different loader and preprocessing method.
- We will learn how to ingest, process and retrieve data from different real-world sources using LangChain.
- The same RAG pipeline can be used with any data source after loading and chunking the documents.

12.2 Supported Data Sources

- Local files (PDF, TXT, DOCX, CSV)
- Web pages (URLs)
- Databases (MySQL, PostgreSQL, MongoDB)
- APIs (REST APIs, JSON responses)
- Cloud storage (S3, Google Drive)
- Structured data (CSV, Excel)
- Unstructured data (PDF, HTML, DOCX)

12.3 Loading PDF Documents

- PDF is a common data source for RAG.
- We can use PyPDFLoader or UnstructuredPDFLoader from LangChain.

```
from langchain.document_loaders import PyPDFLoader
loader = PyPDFLoader("hr_policy.pdf")
documents = loader.load()
print(f"Total pages: {len(documents)}")
```

12.4 Loading Text Files

- Text files are easy to load and process.
- Useful for logs, notes or documentation.

```
from langchain.document_loaders import TextLoader
loader = TextLoader("company_policy.txt")
documents = loader.load()
```

12.5 Loading Web Pages (URL)

- We can load content directly from a website.
- Useful for public documentation or knowledge bases.

```
from langchain.document_loaders import WebBaseLoader
loader = WebBaseLoader("https://example.com")
documents = loader.load()
```

12.6 Loading CSV / Excel Files

- CSV and Excel files contain structured data.
- We can convert rows into text format for embedding.

```
from langchain.document_loaders import CSVLoader
loader = CSVLoader("employee_data.csv")
documents = loader.load()
```

```
from langchain.document_loaders import UnstructuredExcelLoader
loader = UnstructuredExcelLoader("employees.xlsx")
documents = loader.load()
```

12.7 Loading Data from a Database

- We can load data from databases like MySQL, PostgreSQL or MongoDB.
- Useful for enterprise applications where data is stored in databases.

```
from langchain.document_loaders import SQLDatabaseLoader
from langchain.utilities import SQLDatabase

db = SQLDatabase.from_uri("mysql+pymysql://user:password@localhost:3306/hrdb")
loader = SQLDatabaseLoader(
    db,
    "SELECT id, policy_name, description FROM hr_policies",
)
documents = loader.load()
```

12.8 Loading Data from APIs

- We can fetch data from REST APIs and convert the response into documents.
- Useful when data is available via internal or external APIs.

```
import requests
from langchain.schema import Document

url = "https://api.example.com/policies"
response = requests.get(url)
data = response.json()

documents = [
    Document(page_content=str(item), metadata={"source": "api"})
    for item in data
]
```

12.9 Combining Multiple Data Sources

- We can load and combine documents from different sources into a single list.
- This allows the RAG system to search across all sources.

```
all_documents = []
# Load PDF
all_documents.extend(pdf_documents)
# Load Website
all_documents.extend(web_documents)
# Load Database
all_documents.extend(db_documents)
# Load API
all_documents.extend(api_documents)
```

12.10 Complete Example: Multi-Source RAG Pipeline

```
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.embeddings import OpenAIEmbeddings
from langchain.vectorstores import Qdrant

# 1. Load documents from multiple sources
all_documents = pdf_documents + web_documents + db_documents

# 2. Split documents
text_splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50)
chunks = text_splitter.split_documents(all_documents)

# 3. Create embeddings and store in Qdrant
embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
vector_store = Qdrant.from_documents(
      documents=chunks,
      embedding=embeddings,
      url="http://localhost:6333",
  )   collection_name="multi_source_rag"
```

12.11 Best Practices

- Choose the right loader based on data type.
- Clean and preprocess the data before chunking.
- Add meaningful metadata (source, type, date, etc.).
- Combine multiple data sources for better coverage.
- Use hybrid retrieval for structured and unstructured data.
- Keep data updated and refresh embeddings when needed.

12.12 Key Takeaways

- RAG can work with many different data sources.
- Each source requires a specific loader and preprocessing method.
- We can combine multiple sources to build a unified knowledge base.
- The same RAG pipeline can be extended to real-world enterprise use.

Rahul Yadav

13. Advanced RAG Architectures

13.1 Overview

- Basic RAG works well for simple use cases, but for real-world applications we need advanced architectures.
- Advanced RAG architectures improve retrieval quality, scalability, and handle complex data and queries.
- In this section, we will learn different RAG architectures and patterns used in production systems.

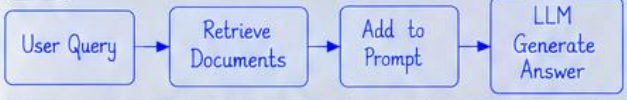
13.2 Types of RAG Architectures

1. Naive RAG - Simple retrieve and generate.
2. Modular RAG - Separate components for flexibility.
3. Agentic RAG - Use agents to plan, retrieve and reason.
4. Graph RAG - Use knowledge graphs for structured data.
5. Multi-Modal RAG - Handle text, images, and other media.
6. Hierarchical RAG - Multi-level retrieval (summary + detail).
7. Self-RAG - LLM decides when and what to retrieve.
8. Hybrid RAG - Combine multiple retrieval techniques.

13.3 Naive RAG Architecture

- The simplest form of RAG.
- Retrieve relevant documents and pass them to the LLM.
- Works well for basic use cases.

Flow:



13.4 Modular RAG Architecture

- Separates the system into independent modules.
- More flexible and easier to maintain.
- Each module can be improved or replaced.

Components:

1. Document Ingestion Module
2. Embedding and Vector Storage Module
3. Retrieval Module
4. Reranking Module
5. Prompt Builder Module
6. LLM Generation Module

13.5 Agentic RAG Architecture

- Uses an agent to plan and decide which tools to use (e.g., search, database, API, calculator).
- The agent can perform multiple retrieval steps, refine the query, and combine results.
- Useful for complex queries and multi-hop questions.

Example (LangChain Agent)

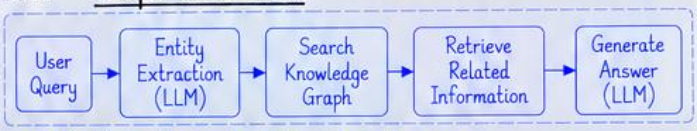
```
from langchain.agents import initialize_agent, Tool
from langchain.chat_models import ChatOpenAI
from langchain.agents import AgentType

llm = ChatOpenAI(model="gpt-4o", temperature=0)
tools = [retriever_tool, search_tool, db_tool]
agent = initialize_agent(
        tools=tools,
        llm=llm,
        agent=AgentType.OPENAI_FUNCTIONS,
        verbose=True
)
```

13.6 Graph RAG Architecture

- Uses a knowledge graph to store entities and relationships.
- Helps in retrieving structured and relational information.
- Useful for domains like enterprise data, research, and knowledge bases.

13.7 Graph RAG Flow



- Identify entities and relations from the query.
- Search the knowledge graph for relevant nodes.
- Retrieve connected information and generate the answer.

13.8 Multi-Modal RAG Architecture

- Handles multiple data types like text, images, tables and PDFs.
- Use separate embedding models for different modalities.
- Combines information from different sources to generate answers.

Example:
Use text embeddings for documents and image embeddings for images) and store them in the same vector database.

13.9 Hierarchical RAG Architecture

- Uses multi-level retrieval.
- First retrieve high-level summaries, then detailed chunks.
- Helps in handling large documents and improving retrieval accuracy.

Flow:



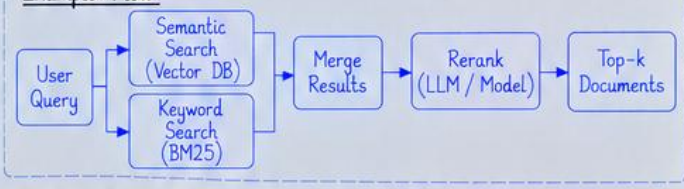
13.10 Self-RAG Architecture

- The LLM decides when to retrieve documents and when to use its own knowledge.
- It can generate follow-up queries to improve retrieval.
- Useful for ambiguous or complex questions.

13.11 Hybrid RAG Architecture

- Combines multiple retrieval techniques for better results.
- Common combination: Semantic Search + Keyword Search (BM25)
- Can also combine multiple vector databases or external search.

Example Flow:



13.12 Production-Ready RAG Architecture

- Includes all best practices for real-world deployment.
- Supports scaling, monitoring and observability.
- Handles large document collections and high traffic.

Components:

1. Data Ingestion Pipeline (Batch/Streaming)
2. Vector Database (Qdrant, Pinecone, etc.)
3. Retrieval Service (Hybrid Search)
4. Reranking and Post-processing
5. LLM Service
6. Caching (Redis)
7. Monitoring and Logging (Prometheus, Grafana, etc.)
8. API Layer (FastAPI / Spring Boot)

13.13 Key Takeaways

- Different RAG architectures are useful for different use cases.
- We can choose the right architecture based on data type, complexity and performance needs.
- Advanced architectures like Agentic, Graph and Multi-Modal RAG provide better accuracy and flexibility.
- Modular and production-ready architectures are important for real-world systems.

Rahul Yadav

14. RAG with Tools and Agents

14.1 Overview

- RAG with tools and agents extends traditional RAG by allowing the LLM to use external tools when needed.
- Tools can be APIs, databases, calculators, web search, or custom functions.
- Agents decide which tools to use, in what order, and how to combine results to generate the final answer.
- This approach is useful for complex, real-world applications where a single retrieval step is not enough.

14.2 Why Use Tools and Agents in RAG ?

- Handle complex multi-step questions.
- Fetch real-time information from external sources.
- Perform calculations or data analysis.
- Interact with external APIs and databases.
- Improve accuracy and provide more complete answers.

14.3 Types of Tools

- Search Tools – Web search (e.g., Tavily, SerpAPI)
- Database Tools – Query databases (SQL, NoSQL)
- API Tools – Call external REST APIs
- Calculator Tools – Perform mathematical calculations
- Custom Tools – Domain-specific functions
- File Tools – Read/write/parse files (PDF, CSV, etc.)

14.4 What is an Agent ?

- An agent is an LLM-based system that can:
  - Understand the user's goal
  - Decide which tools to use
  - Plan and execute multiple steps
  - Combine results to generate the final answer
- Agents use techniques like ReAct (Reasoning + Acting) to think, act, observe and refine the answer.

14.5 Agent Workflow (ReAct Pattern)



- Agent first analyzes the query and decides which tools are needed.
- It may use multiple tools step by step.
- It observes the results and refines the answer if needed.
- Finally, it generates a complete and accurate response.

14.6 Example: RAG with Tools (LangChain)

```
from langchain.agents import initialize_agent, Tool
from langchain.chat_models import ChatOpenAI
from langchain.tools import TavilySearchResults, PythonREPLTool
from langchain.vectorstores import Qdrant
from langchain.embeddings import OpenAIEmbeddings
from langchain.chains import RetrievalQA

# Initialize tools
search_tool = TavilySearchResults(max_results=3)
python_tool = PythonREPLTool()

tools = [
    Tool(name="Search", func=search_tool.run,
         description="Useful for searching the web"),
    Tool(name="Python", func=python_tool.run,
         description="Useful for calculations and data analysis")
]
```

14.7 Complete RAG Agent Example

```
# Initialize LLM
llm = ChatOpenAI(model="gpt-4o", temperature=0)
# Initialize vector store and retriever
embeddings = OpenAIEmbeddings()
vector_store = Qdrant.from_documents(
    documents, embeddings,
    url="http://localhost:6333",
    collection_name="rag_docs"
)
retriever = vector_store.as_retriever(search_kwargs={"k": 4})
# Create retrieval tool
retrieval_tool = Tool(
    name="DocumentSearch",
    func=lambda query: retriever.get_relevant_documents(query),
    description="Searches internal documents for relevant information"
)
# Initialize agent
agent = initialize_agent(
    tools=[retrieval_tool,search_tool, python_tool],
    llm=llm,
    agent="zero-shot-react-description",
    verbose=True
)
# Run agent
query = "What is our leave policy and also search latest HR update?"
response = agent.run(query)
print(response)
```

14.8 Example Use Cases

- HR Assistant – Search policies (internal docs) + fetch latest updates from web + answer queries.
- Customer Support + Retrieve internal knowledge + call API for real-time order status.
- Finance Assistant – Get company reports (PDF) + perform calculations + generate summary.
- Research Assistant – Search academic papers + summarize findings.
- IT Support – Retrieve troubleshooting docs + check system status via API.

14.9 Advanced Agent Techniques

- Tool Routing – Automatically select the best tool for a query.
- Multi-step Reasoning – Break complex questions into smaller steps.
- Memory – Keep conversation history for follow-up questions.
- Self-correction – Validate results and refine if needed.
- Planning – Use planning agents to decide the sequence of actions.
- Function Calling – Use structured tool calls with JSON output.

14.10 Best Practices

- Clearly define tool descriptions so the agent can use them correctly.
- Use reliable and relevant tools.
- Set limits on the number of tool calls to avoid loops.
- Validate and filter tool outputs before passing to the LLM.
- Use structured outputs (JSON) for better control.
- Monitor and log agent actions for debugging.
- Handle errors and fallback gracefully.
- Keep tool results concise and relevant to the query.

Key Takeaways

- Tools and agents make RAG more powerful and flexible.
- Agents can handle complex, real-world queries using multiple tools.
- Use LangChain agents with tools like search, database, API, and calculator.
- Follow best practices to ensure accurate, reliable and safe responses.

Rahul Yadav

15. Production Deployment of RAG

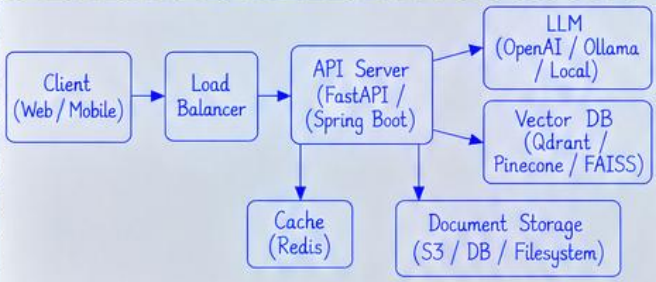
15.1 Overview

- Deploying RAG in production requires careful planning for scalability, reliability, security and monitoring.
- A production-ready RAG system should handle real users, large data, and unexpected failures.
- We will learn the architecture, deployment options, scaling strategies and best practices for real-world use.

15.2 Key Requirements

- High availability and low latency.
- Scalable document ingestion and retrieval.
- Secure access and data protection.
- Caching to improve performance.
- Monitoring, logging and observability.
- Easy updates for documents and models.

15.3 High Level Architecture



15.4 Technology Stack Options

- Backend – FastAPI (Python) / Spring Boot (Java)
- LLM – OpenAI / Azure OpenAI / Ollama (Self-hosted)
- Vector DB – Qdrant / Pinecone / Weaviate / FAISS
- Document Storage – AWS S3 / Azure Blob / Local Storage
- Cache – Redis
- Containerization – Docker
- Orchestration – Kubernetes
- Monitoring – Prometheus + Grafana
- Logging – ELK Stack / Loki
- CI/CD – GitHub Actions / Jenkins

15.5 Building a Production API (FastAPI)

```
from fastapi import FastAPI, Request
from pydantic import BaseModel
from langchain.chains import RetrievalQA
from langchain_openai import ChatOpenAI
from langchain_community.vectorstores import Qdrant
from langchain.embeddings import OpenAIEmbeddings

app = FastAPI()

# Initialize components
llm = ChatOpenAI(model="gpt-4o", temperature=0)
embeddings = OpenAIEmbeddings()
vector_store = Qdrant.from_existing_collection(
    url="http://localhost:6333",
    collection_name="rag_docs",
    embedding=embeddings)
)
retrieval_chain = RetrievalQA.from_chain_type(
    llm=llm,
    retriever=vector_store.as_retriever(search_kwargs={"k": 4})
)
class QueryRequest(BaseModel):
    query: str

@app.post("/query")
async def query_rag(request: QueryRequest):
    result = retrieval_chain.invoke({"query": request.query})
    return {"answer": result["result"]}
```

15.6 Docker Deployment

```
# Dockerfile
FROM python:3.10-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

15.7 requirements.txt (Example)

```
fastapi
uvicorn
langchain
langchain-openai
langchain-community
qdrant-client
redis
python-dotenv
```

15.8 Kubernetes Deployment (Concept)

- Use Kubernetes for automatic scaling and high availability.
- Deploy API server, vector DB and cache as separate services.
- Use Kubernetes HPA (Horizontal Pod Autoscaler) to scale based on CPU/memory usage.
- Use persistent volumes for vector DB data.

Example Kubernetes Components:

- Deployment – For API server
- Service – To expose the API
- ConfigMap – For environment variables
- Secret – For API keys
- PersistentVolume – For vector database storage
- Ingress – To route external traffic
- HPA – To scale based on load

15.9 Caching Strategy (Redis)

- Cache embeddings for frequently asked queries.
- Cache final answers to reduce LLM calls.
- Use Redis with TTL (time to live) for automatic expiration.

```
import redis
import json
redis_client = redis.Redis(host="localhost", port=6379, db=0)

# Get cached answer
cached = redis_client.get(query)
if cached:
    print("Returning cached answer")
    return json.loads(cached)

# After generating answer
redis_client.setex(query, 3600, json.dumps(result))
```

15.10 Monitoring and Observability

- Use Prometheus + Grafana to monitor API latency, error rate, CPU and memory usage.
- Log important events (queries, errors, response time) using ELK or Loki.
- Monitor vector DB performance and disk usage.
- Set up alerts for failures or high latency.

15.11 Best Practices

- Use streaming response for better user experience.
- Implement rate limiting to avoid abuse.
- Keep API keys and sensitive data in environment variables or secrets.
- Use retry and fallback mechanisms for LLM and vector DB.
- Regularly update documents and re-generate embeddings.
- Test with real user load before full deployment.

15.12 Key Takeaways

- Production deployment requires a scalable, secure and reliable architecture.
- Use modern tools like Docker, Kubernetes, Redis, and monitoring tools.
- Caching, logging, and proper monitoring are essential.
- Follow best practices to ensure a stable and cost-effective RAG system.

Rahul Yadav

16. Practical Projects and Use Cases

16.1 Overview

- RAG can be applied in many real-world scenarios across different industries.
- In this section, we will explore practical projects and real use cases to understand how to build end-to-end RAG systems.
- Each project will cover data ingestion, storage, retrieval, and generation with a complete example.

16.2 Project 1: PDF Q&A Chatbot

- This project allows users to ask questions from PDF documents (e.g., company policy, technical docs).
- We will use LangChain, OpenAI embeddings and Qdrant.

```
from langchain.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.embeddings import OpenAIEmbeddings
from langchain.vectorstores import Qdrant
from langchain.chains import RetrievalQA
from langchain.chat_models import ChatOpenAI

loader = PyPDFLoader("company_policy.pdf")
docs = loader.load()

splitter = RecursiveCharacterTextSplitter(chunk_size=500,
                                          chunk_overlap=50)
chunks = splitter.split_documents(docs)

embeddings = OpenAIEmbeddings()
vector_store = Qdrant.from_documents(chunks, embeddings,
                                     url="http://localhost:6333",
                                     collection_name="pdf_chatbot")

qa_chain = RetrievalQA.from_chain_type(
    llm=ChatOpenAI(model="gpt-4o"),
    retriever=vector_store.as_retriever()
)
query = "What is the leave policy?"
result = qa_chain.run(query)
print(result)
```

16.3 Project 2: Website Q&A Bot

- This project loads content from a website and allows users to ask questions.
- Useful for documentation, blogs or knowledge bases.

```
from langchain.document_loaders import WebBaseLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.embeddings import OpenAIEmbeddings
from langchain.vectorstores import Qdrant
from langchain.chains import RetrievalQA

loader = WebBaseLoader("https://example.com")
docs = loader.load()
splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50)
chunks = splitter.split_documents(docs)

embeddings = OpenAIEmbeddings()
vector_store = Qdrant.from_documents(chunks, embeddings,
                                     url="http://localhost:6333",
                                     collection_name="website_bot")

qa_chain = RetrievalQA.from_chain_type(
    llm=ChatOpenAI(model="gpt-4o"),
    retriever=vector_store.as_retriever()
)
print(qa_chain.run("What services does this website provide?"))
```

16.4 Project 3: Querying a Database (Text to SQL)

- This project converts natural language questions into SQL queries and fetches results from a database.
- Useful for business analytics and data analysis.

```
from langchain.utilities import SQLDatabase
from langchain.chains import SQLDatabaseChain
from langchain.chat_models import ChatOpenAI

# Connect to database
db = SQLDatabase.from_uri("mysql+pymysql://user:password@localhost:3306/hrdb")

# Create chain
db_chain = SQLDatabaseChain.from_llm(
    llm=ChatOpenAI(model="gpt-4o"),
    db=db,
    verbose=True
)
# Ask question
query = "Show total number of employees in each department"
result = db_chain.run(query)
print(result)
```

16.5 Project 4: RAG with APIs and External Tools

- This project combines RAG with external APIs (e.g., weather, stock prices, HR systems) to provide real-time answers.
- We will use LangChain agents and tools.

```
from langchain.agents import initialize_agent, Tool
from langchain.chat_models import ChatOpenAI
import requests

# Define a tool to get weather information
def get_weather(city: str):
    url = f"https://api.openweathermap.org/data/2.5/weather?q={city}&appid=API_KEY"
    response = requests.get(url).json()
    return response["weather"][0]["description"]

weather_tool = Tool(
    name="Weather",
    func=get_weather,
    description="Get current weather information for a city"
)
llm = ChatOpenAI(model="gpt-4o")
agent = initialize_agent(
    tools=[weather_tool],
    llm=llm,
    agent="zero-shot-react-description",
    verbose=True
)
print(agent.run("What is the weather in Delhi and also
                 summarize latest company policy from docs?"))
```

16.6 Other Real-World Use Cases

- Customer Support Bot – Answer FAQs from internal docs.
- HR Assistant – Employee policies, leave, benefits, etc.
- Finance Assistant – Analyze reports and generate summaries.
- Research Assistant – Search academic papers and explain findings.
- Healthcare Assistant – Use medical documents for diagnosis support.
- Legal Assistant – Search legal documents and case laws.
- Education Tutor – Answer questions from study materials.

16.7 Best Practices for Real-World Projects

- Choose the right data source and preprocessing strategy.
- Use proper chunking and metadata.
- Apply hybrid retrieval and re-ranking for better accuracy.
- Add caching for frequently asked questions.
- Implement security, authentication and access control.
- Monitor performance and user feedback.
- Continuously update documents and embeddings.

16.8 Key Takeaways

- RAG can be applied in many industries and real-world scenarios.
- Each use case may require different data sources and tools.
- Combining retrieval with APIs, databases and agents makes RAG more powerful.
- Real-world projects help in understanding practical challenges and solutions.

Rahul Yadav

17. Common Challenges and Best Practices

17.1 Overview

- While building RAG systems, we face several challenges in real-world applications.
- In this section, we will discuss common challenges, solutions and best practices to build robust and production-ready RAG systems.
- Following these practices helps in better accuracy, reliability, security and performance.

17.2 Challenge 1: Poor Retrieval Accuracy

- The retrieved documents may not be relevant to the query.
- This leads to incorrect or hallucinated answers.
- Reasons: small chunks, wrong embedding model, no metadata, or poor vector search configuration.

Solutions and Best Practices:

- Use a good embedding model (e.g., text-embedding-3-small).
- Use proper chunking strategy (500-1000 tokens with overlap).
- Add metadata (source, title, date, category).
- Use hybrid search (vector + keyword).
- Apply re-ranking to get more relevant results.

17.3 Challenge 2: Hallucinations in Answers

- The LLM may generate incorrect or made-up information that is not present in the documents.
- This happens when the model relies on its own knowledge instead of the retrieved context.

Solutions and Best Practices:

- Provide clear system prompt to use only retrieved context.
- Include source citations in the answer.
- Use query rewriting to improve retrieval.
- Use re-ranking and relevance filtering.
- Use models with lower hallucination rate (e.g., GPT-4).

17.4 Challenge 3: Handling Large Documents

- Large documents may not fit into the model context window.
- It is difficult to retrieve relevant information from long documents.

Solutions and Best Practices:

- Split documents into smaller chunks.
- Use hierarchical chunking (parent-child).
- Store summary along with detailed chunks.
- Use multi-stage retrieval (summary → detailed chunk).
- Use structured data extraction for better indexing.

17.5 Challenge 4: Handling Different File Formats

- RAG systems need to support multiple formats like PDF, DOCX, TXT, CSV, Excel, HTML, etc.
- Each format requires different processing techniques.

Solutions and Best Practices:

- Use appropriate document loaders from LangChain.
- Clean and preprocess data before indexing.
- Extract text, tables and metadata properly.
- Handle images using OCR (e.g., Tesseract, OpenAI Vision).
- Convert structured data (CSV, Excel) to text or JSON format.

17.6 Challenge 5: Performance and Latency

- RAG systems can be slow due to document retrieval, embedding generation and LLM inference.
- Real-time applications require low latency responses.

Solutions and Best Practices:

- Use caching (Redis) for embeddings and final answers.
- Precompute and store embeddings.
- Use efficient vector database (Qdrant, Pinecone, etc.).
- Use smaller models for simple queries.
- Optimize chunk size and retrieval top-k values.

17.7 Challenge 6: Scalability

- As data grows, the system needs to handle large document collections and high user traffic.
- Embedding generation, storage and retrieval can become costly.

Solutions and Best Practices:

- Use scalable vector databases (Qdrant, Pinecone, Weaviate).
- Implement distributed architecture (Kubernetes).
- Use batch processing for document ingestion.
- Use load balancing and auto-scaling.
- Monitor system performance and optimize regularly.

17.8 Challenge 7: Security and Privacy

- Documents may contain sensitive or confidential information.
- We need to ensure data security and access control.

Solutions and Best Practices:

- Implement authentication and authorization.
- Use data encryption (at rest and in transit).
- Restrict access to documents based on user roles.
- Remove or mask sensitive data during ingestion.
- Use private deployment (on-premise or VPC) for sensitive data.

17.9 Challenge 8: Keeping Data Updated

- Documents and data are frequently updated.
- The RAG system should always use the latest information.

Solutions and Best Practices:

- Implement scheduled data ingestion pipelines.
- Use incremental updates instead of reprocessing all data.
- Remove old embeddings when documents are deleted.
- Add versioning and metadata (last updated date).
- Automate the pipeline using tools like Airflow or cron jobs.

17.10 Challenge 9: Cost Management

- Embedding generation and LLM inference can be expensive.
- Large document collections and high user traffic increase costs.

Solutions and Best Practices:

- Use efficient embedding models (e.g., text-embedding-3-small).
- Cache embeddings and responses.
- Use smaller or open-source models for simple queries.
- Implement query routing (use simple model for general questions).
- Monitor API usage and set cost limits.

17.11 Challenge 10: Monitoring and Evaluation

- It is difficult to measure the quality and performance of RAG systems in production.
- We need to track accuracy, latency, errors and user feedback.

Solutions and Best Practices:

- Use evaluation metrics (Precision@K, Recall@K, MRR, RAGAS).
- Monitor system logs and performance metrics.
- Track user feedback and query patterns.
- Set up alerts for failures or high latency.
- Regularly test with real user queries and update the system.

17.12 Key Takeaways

- RAG systems face challenges in retrieval accuracy, hallucinations, scalability, security, performance and cost.
- We can solve these challenges by following best practices like proper chunking, hybrid search, caching, monitoring and evaluation.
- A production-ready RAG system should be scalable, secure, efficient and regularly updated.
- Continuous improvement and real-world testing are important for long-term success.

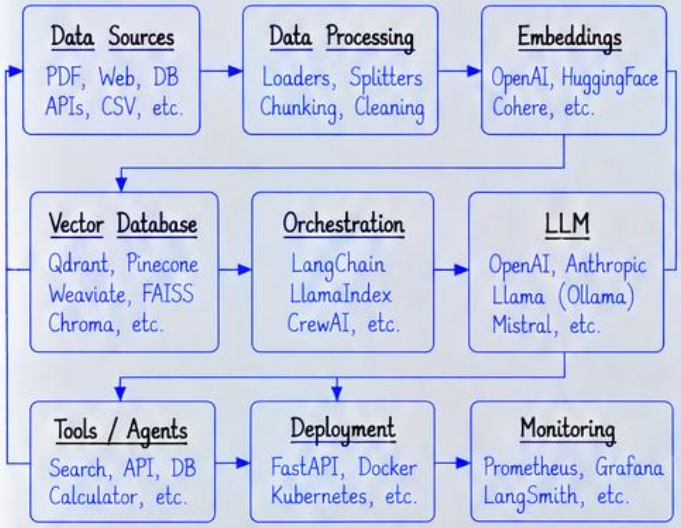
Rahul Yadav

18. Tools and Frameworks Ecosystem

18.1 Overview

- Building a RAG system involves multiple tools and frameworks for document loading, text processing, embeddings, vector storage, LLMs, orchestration, deployment, monitoring and evaluation.
- In this section, we will explore the popular tools used in each component and when to use them.

18.2 RAG Ecosystem Overview



18.3 Document Loading Tools

- Used to load data from different sources like PDF, websites, Databases, APIs, etc.
- LangChain provides many ready-to-use document loaders.

Popular Document Loaders:

- PyPDFLoader – Load PDF documents
- WebBaseLoader – Load data from websites
- CSVLoader – Load CSV files
- UnstructuredLoader – Load various file formats (DOCX, PPT, etc.)
- DirectoryLoader – Load multiple files from a folder
- NotionLoader – Load data from Notion
- HTMLLoader – Load HTML files
- SQLDatabaseLoader – Load data from databases

18.4 Text Processing and Chunking Tools

- Used to split documents into smaller chunks, clean text and preprocess data.
- Good chunking improves retrieval accuracy.

Popular Tools:

- RecursiveCharacterTextSplitter (LangChain)
- TokenTextSplitter (LangChain)
- NLTK / SpaCy – Text cleaning, tokenization, stopwords
- Unstructured – Advanced document parsing
- Tesseract – OCR for images and scanned documents

18.5 Embedding Models and Libraries

- Embeddings convert text into vector representations so that similar documents can be retrieved.
- We can use OpenAI, HuggingFace, Cohere and other models.

Popular Embedding Models:

- OpenAIEmbeddings (text-embedding-3-small, -large)
- HuggingFaceEmbeddings (sentence-transformers)
- CohereEmbeddings
- OllamaEmbeddings (open source models like nomic-embed)
- InstructorEmbeddings (good for domain specific data)

18.6 Vector Databases

- Used to store and search embeddings efficiently.
- Helps in similarity search to retrieve relevant documents.
- Choose based on scalability, features and deployment needs.

Popular Vector Databases:

- Qdrant – Open source, high performance, easy to deploy
- Pinecone – Managed vector database (cloud)
- Weaviate – Open source, supports hybrid search
- FAISS – Facebook's library (good for small/medium scale)
- Chroma – Simple and lightweight (good for prototypes)
- Milvus – Scalable and production ready
- Atlas Vector Search – MongoDB integration

18.7 LLMs (Large Language Models)

- LLMs are used to understand the query and generate the final answer using retrieved documents.
- We can use cloud-based or open source models.

Popular LLMs:

- OpenAI – GPT-4o, GPT-4, GPT-3.5
- Anthropic – Claude 3
- Meta – Llama 3 (open source)
- Mistral – Mistral 7B, Mixtral
- Google – Gemini
- Ollama – Run open source models locally (Llama, Mistral, etc.)

18.8 Orchestration Frameworks

- Used to build the RAG pipeline, manage chains, agents, retrieval and tool integration.

Popular Frameworks:

- LangChain – Most popular, flexible and easy to use
- LlamaIndex – Good for data indexing and retrieval
- CrewAI – Multi-agent framework
- Semantic Kernel – By Microsoft
- Haystack – Modular and production ready
- AutoGen – Multi-agent conversations (by Microsoft)

18.9 Tools for Agents and External Integration

- Agents can use external tools like web search, APIs, databases, calculators, etc. to get real-time information.

Popular Tools:

- Tavily / SerpAPI – Web search
- Requests / HTTP tools – Call external APIs
- SQL Database tools – Query databases
- Python REPL – Run Python code
- Calculator – Perform mathematical calculations
- Custom Tools – Define your own functions

18.10 Deployment and Infrastructure Tools

- Used to deploy and scale RAG systems in production.
- Helps in containerization, orchestration and scaling.

Popular Tools:

- FastAPI – Build REST APIs
- Docker – Containerization
- Kubernetes – Orchestration and auto-scaling
- AWS / Azure / GCP – Cloud deployment
- Nginx – Reverse proxy and load balancing
- CI/CD – GitHub Actions, Jenkins

18.11 Monitoring, Evaluation and Observability

- Used to monitor performance, logs, errors and user feedback.
- Helps in improving the RAG system over time.

Popular Tools:

- LangSmith – Tracing, debugging and evaluation (by LangChain)
- Prometheus + Grafana – Monitoring and alerting
- ELK Stack (Elasticsearch, Logstash, Kibana) – Log management
- RAGAS – Evaluate RAG performance (precision, recall, etc.)
- Weights & Biases – Experiment tracking
- OpenTelemetry – Distributed tracing

19. Interview Questions and Final Revision

19.1 Important Interview Topics

- RAG concepts and architecture
- Document loading, chunking and embeddings
- Vector databases (Qdrant, Pinecone, Weaviate)
- Prompt engineering and retrieval strategies
- LangChain, LlamaIndex and agent frameworks
- Evaluation metrics and optimization
- Production deployment, scaling and monitoring
- Real-world use cases and best practices

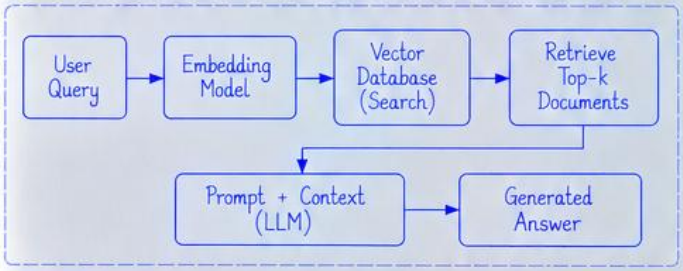
19.2 Frequently Asked Interview Questions (Basic)

Q1. What is RAG ?
RAG (Retrieval Augmented Generation) is a technique that combines information retrieval with LLMs. It retrieves relevant documents from an external data source and provides them to the LLM as context to generate more accurate and up-to-date answers.

Q2. How does RAG work ?
It converts the query into an embedding, searches similar documents in a vector database, retrieves the top-k relevant chunks, and passes them to the LLM along with the prompt to generate the final answer.

Q3. Why is RAG better than fine-tuning ?
RAG does not require model training, can use latest data, is cost-efficient, and reduces hallucinations. Fine-tuning is useful for domain-specific tasks but requires labeled data and higher compute cost.

19.3 RAG Architecture (Interview Explanation)



19.4 Intermediate Interview Questions

Q4. What is the role of embeddings in RAG ?
Embeddings convert text into numerical vectors so that similar content can be found using vector similarity search.

Q5. What are the different chunking strategies ?
Fixed size, recursive, semantic, document structure based, and domain-specific chunking.

Q6. Which vector database have you used ?
Qdrant. It supports high performance similarity search, filtering, metadata, and is easy to deploy using Docker.

Q7. How do you handle large documents ?
Split into smaller chunks, use hierarchical summarization, store metadata, and use parent-child or multi-stage retrieval.

Q8. How do you reduce hallucinations ?
Use relevant retrieved context, re-ranking, proper prompting, citation of sources, and filtering low-similarity results.

19.5 Advanced Interview Questions

Q9. What is hybrid search ?
Hybrid search combines vector search (semantic) with keyword search (e.g., BM25) to improve retrieval accuracy,

Q10. How do you evaluate a RAG system ?
Metrics: Precision@K, Recall@K, MRR, RAGAS (faithfulness, answer relevancy, context recall), and user feedback.

Q11. Explain RAG with agents.
Agents can use tools (search, API, database, calculator, etc.) decide which tool to use, retrieve relevant data and generate final answers in a multi-step workflow.

Q12. How do you handle real-time or updated data ?
Use incremental ingestion, scheduled pipelines, or real-time indexing with tools like Airflow. Also use metadata with timestamps and remove old data.

Q13. How would you deploy a RAG system in production ?
Use FastAPI, Docker, Kubernetes, vector DB (Qdrant), monitoring (Prometheus + Grafana), caching (Redis), and logging (ELK). Ensure proper security and scaling.

Q14. What are the key challenges in production RAG ?
Retrieval accuracy, hallucinations, latency, scalability, security, cost management and keeping data updated.

19.6 Code Related Interview Questions

Q15. How do you load a PDF in LangChain ?
Use PyPDFLoader, then split into chunks and create embeddings.

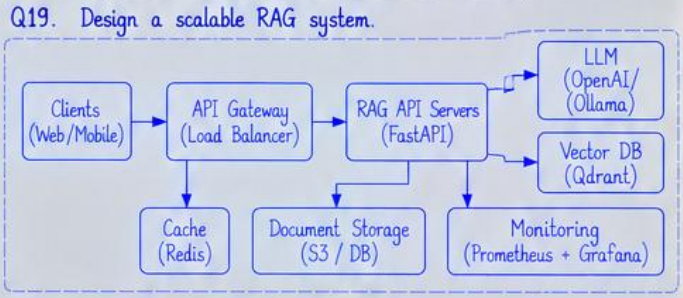
Q16. How do you store documents in Qdrant ?
Create embeddings and use Qdrant vector store to store text along with metadata.

Q17. How do you implement a retrieval chain ?
Use RetrievalQA or create a custom chain with retriever and LLM.

Q18. How do you implement a RAG API using FastAPI ?
Create an endpoint, accept user query, run retrieval chain and return the generated answer as JSON response.

19.7 System Design Interview Question

Q19. Design a scalable RAG system.



19.8 Final Revision Checklist

- Understand end-to-end RAG pipeline.
- Practice common and advanced interview questions.
- Be comfortable with code examples (document loading, retrieval, API, deployment).
- Revise key concepts: embeddings, vector DB, chunking, evaluation.
- Go through real-world project examples.
- Understand trade-offs, best practices and production considerations.
- Review system design and scalability.
- Be ready to explain your own projects and experiences.

Rahul Yadav